

Bases de Datos

Clase 21: Two-Phase Locking, Deadlocks y Triggers

Miércoles 13 de mayo

Recordatorio de la clase anterior

- Anomalías de concurrencia: dirty / non-repeatable / lost update / phantom.
- Serializabilidad y grafo de precedencia (acíclico $\Leftrightarrow$ conflict-serializable).
- **Locks S y X, y Two-Phase Locking** (dos fases: crecimiento + decrecimiento).
- 2PL $\Rightarrow$ todo schedule resultante es conflict-serializable.

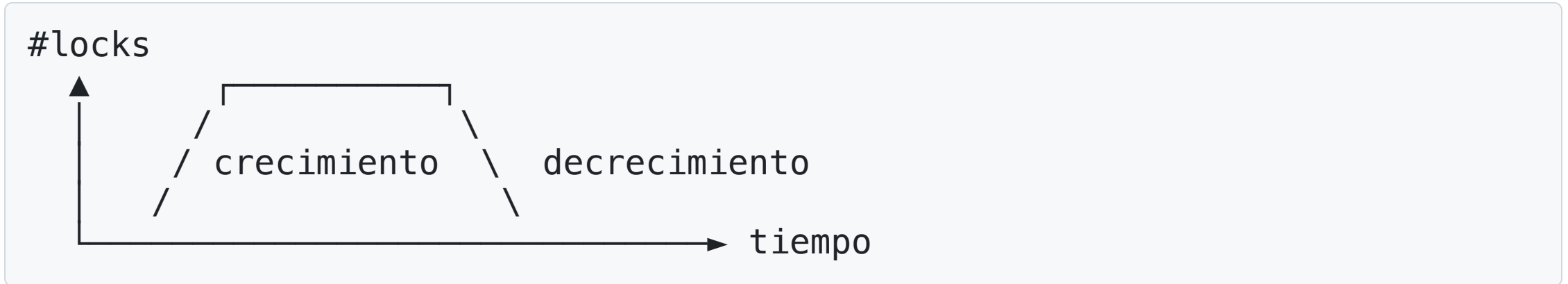
Hoy: dónde 2PL no basta y la realidad es más sutil — Strict 2PL, deadlocks, niveles de aislamiento (con sus trampas en producción) y triggers.

Agenda

1. **Strict 2PL**: por qué liberar locks sólo en el commit.
2. **Deadlocks**: detección y prevención.
3. **Niveles de aislamiento**.
4. **Triggers**: reglas activas dentro de la transacción.

Strict 2PL

Recordatorio: 2PL puro



- **Crecimiento:** sólo adquiere locks.
- **Decrecimiento:** sólo libera locks.

→ Garantiza schedules conflict-serializables.

Pero — ¿qué pasa si una transacción libera un lock **antes** de hacer commit, y después aborta?

El problema: cascading aborts

```
T1: X(A) W(A,5) release(A) ..... ROLLBACK
T2:           X(A) R(A)→5 W(A,10) release(A) ..... commit?
T3:           S(A) R(A)→10 ..... commit?
```

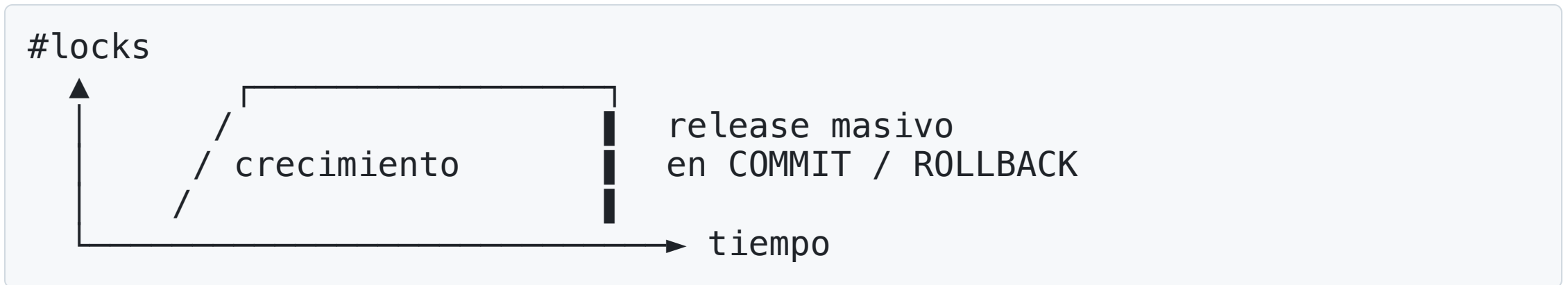
- T1 escribió `A=5` , liberó el lock, y **luego abortó**.
- T2 alcanzó a leer `A=5` , calculó `A=10` , lo escribió, y liberó.
- T3 leyó `A=10` , valor que existe **sólo porque T1 escribió antes de abortar**.

T1 aborta → T2 trabajó con un valor que nunca debió existir → **T2 debe abortar** → pero T3 ya leyó la salida de T2 → **T3 también debe abortar** → ...

Una sola transacción que falla **encadena** aborts. Es correcto, pero muy caro: pierdes trabajo de transacciones que jamás tocaron a T1 directamente.

Solución: Strict 2PL

Strict 2PL: todos los locks se liberan **en el commit/abort**, nunca antes.



- Si T1 aborta, **nadie alcanzó a leer** sus escrituras → no hay cascada.
- Es el **protocolo estándar en la práctica** (PostgreSQL, MySQL, etc).

Strict 2PL: dos reglas

Regla 1. Si una transacción T quiere **leer** (resp. **modificar**) un objeto X , primero pide un **shared lock** (resp. **exclusive lock**) sobre X .

- Si el lock no se puede otorgar (incompatibilidad), T se **suspende** hasta que sea liberado.
- Para obtener $X(A)$, no puede haber **ningún** lock previo sobre A .

Regla 2. Cuando la transacción se completa (**COMMIT** o **ROLLBACK**), **libera todos sus locks de una vez**.

Ejemplo: transferencia bajo Strict 2PL

```
T1 (Alice: AB → C, 100):  
  X(AB); R(AB); W(AB);  
  X(C); R(C); W(C);  
  COMMIT → libera todo
```

```
T2 (Bob: AB → C, 200):  
  pide X(AB) → ESPERA (T1 lo tiene)  
  ...
```

T2 sólo puede empezar a tocar **AB** **después** de que T1 haga commit. → No hay lost update, no hay doble cobro.

El precio: T2 espera. La concurrencia se reduce, pero la corrección se garantiza.

Deadlocks

El precio de 2PL: deadlocks

Dos transferencias entre las **mismas** dos cuentas, pero en sentidos opuestos:

```
T1 (Alice → Bob, 100):      T2 (Bob → Alice, 50):  
  X(Alice) R W              X(Bob)    R W  
  pide X(Bob) ———espera——→ pide X(Alice) ———espera——→
```

- T1 tiene **Alice**, espera **Bob**.
- T2 tiene **Bob**, espera **Alice**.
- **Ambas esperan para siempre.**

Un **deadlock** es un ciclo de transacciones donde cada una espera un lock que otra mantiene.

Forma abstracta del mismo deadlock

T1: X(A) — pide X(B) — ?
T2: X(B) — pide X(A) — ?

Generalización: n transacciones esperándose mutuamente forman un **ciclo** en el grafo de "quién espera a quién".

Detección: el grafo de espera (waits-for graph)

- **Nodos:** transacciones activas.
- **Arista** $T_i \rightarrow T_j$ si T_i espera un lock que T_j mantiene.

Hay deadlock $\Leftrightarrow$ el grafo de espera **tiene un ciclo**.



¿Cuándo se chequea? No es periódico globalmente. PostgreSQL: cuando una transacción se ha quedado bloqueada esperando un lock por más de `deadlock_timeout` (default 1 s), entonces construye el grafo y busca ciclos. Esto evita pagar el costo del chequeo en bloqueos cortos legítimos.

Resolución: elegir una víctima

Cuando se detecta un ciclo, el DBMS:

1. Elige una **víctima** (la más joven, la que menos trabajo ha hecho, ...).
2. **Aborta** la víctima → libera sus locks → el ciclo se rompe.
3. La aplicación recibe un error (`SQLSTATE 40P01` en Postgres, `40001` para SSI) y típicamente **reintenta** la transacción.

En PostgreSQL:

```
ERROR:  deadlock detected
DETAIL: Process 1234 waits for ShareLock on transaction 9876;
        blocked by process 5678.
        Process 5678 waits for ShareLock on transaction 5432;
        blocked by process 1234.
HINT:   See server log for query details.
CONTEXT: while updating tuple (0,5) in relation "cuentas"
```

Timeouts y prevención de deadlocks

PostgreSQL **no usa** prevención — prefieren detectar y abortar. Algunos sistemas previenen deadlocks abortando las que llevan mucho tiempo (u otra política).

Parámetro	Qué dispara
<code>deadlock_timeout</code>	Tras esperar este lock por T segundos, chequea ciclos (no aborta)
<code>lock_timeout</code>	Aborta la transacción si espera un solo lock más de T
<code>statement_timeout</code>	Aborta una sentencia individual si tarda más de T
<code>idle_in_transaction_session_timeout</code>	Cierra una transacción abierta y ociosa (sin emitir queries) más de T

Resumen del control de concurrencia

Estrategia	Idea	Costo
2PL	Locks en dos fases	Posibles deadlocks
Strict 2PL	+ libera al commit	Sin cascading aborts; estándar
Detección	Grafo de espera + víctima	Latencia variable
Timeout	Rendirse tras T segundos	Falsos positivos
Wound-wait / Wait-die	Prioridad por timestamp	Sin deadlocks; más aborts

PostgreSQL combina **Strict 2PL** para escrituras + **MVCC** para lectores + detección de deadlocks al expirar `deadlock_timeout` .

Niveles de Aislamiento

Trade-off: corrección vs. throughput

Strict 2PL "completo" (S y X largos, además de bloquear predicados) garantiza **serializable** — pero es **caro**: más locks → más espera → menos concurrencia.

Muchas aplicaciones **toleran** ciertas anomalías a cambio de throughput.

El estándar SQL define **4 niveles de aislamiento** que corresponden a relajar progresivamente las reglas de bloqueo.

Los 4 niveles del estándar SQL

Nivel	Dirty	Non-Repeat.	Phantom
READ UNCOMMITTED	×	×	×
READ COMMITTED	✓	×	×
REPEATABLE READ	✓	✓	×
SERIALIZABLE	✓	✓	✓

✓ = anomalía **prevenida** ; × = anomalía **posible**.

```
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
```

El modelo clásico: variantes de 2PL

Los niveles se **definieron originalmente** como políticas de duración de locks sobre 2PL (Gray, 1976):

Nivel	Locks de lectura (S)	Locks de escritura (X)	Predicate locks
READ UNCOMMITTED	ninguno	hasta commit	no
READ COMMITTED	cortos (release tras leer)	hasta commit	no
REPEATABLE READ	hasta commit	hasta commit	no
SERIALIZABLE	hasta commit	hasta commit	sí

Esta es la implementación **clásica / de libro de texto**.

Los DBMSs modernos lo implementan distinto.

Por qué REPEATABLE READ $\neq$ Serializable

Ejemplo clásico (Berenson et al., 1995): dos doctores de turno, regla "siempre debe haber al menos uno":

```
T1 (Alice): SELECT count(*) WHERE on_call = TRUE;    -- ve 2
            UPDATE doctors SET on_call=FALSE WHERE id=Alice;  -- yo me voy
T2 (Bob):   SELECT count(*) WHERE on_call = TRUE;    -- ve 2 (mismo snapshot)
            UPDATE doctors SET on_call=FALSE WHERE id=Bob;    -- yo me voy
```

Ambas commit. **Nadie está de turno.** En SERIALIZABLE una de las dos sería abortada, veamos por qué.

Predicate locks

Un **predicate lock** bloquea una *condición* (`on_call = TRUE`, `precio > 100`, ...) en lugar de filas específicas.

- **S sobre predicado P** : nadie puede insertar, borrar o modificar una fila de modo que cambie qué filas satisfacen P , hasta que se libere.
- **X sobre predicado P** : nadie puede leer ni escribir filas que satisfagan P .

→ Previenen phantoms y write skew.

Predicate locks: el ejemplo de los doctores

- T1 lee `WHERE on_call = TRUE` → toma `S` sobre ese predicado.
- T2 lee lo mismo → `S` es compatible con `S`, ambas ven 2.
- T1 hace `UPDATE ... SET on_call = FALSE WHERE id = Alice` → esta escritura **cambia quién satisface `on_call = TRUE`** → necesita `X` sobre el predicado → choca con la `S` de T2.
- Análogo para T2 → **deadlock** → una aborta. Invariante salvado.

El problema: decidir "¿esta escritura afecta a este predicado?" en general es caro. Los DBMS reales **no** los implementan literalmente — usan aproximaciones (next-key locks en InnoDB) o SSI (PostgreSQL).

Aislamiento en el estándar vs 2PL

Nivel	Versión de 2PL
READ UNCOMMITTED	Strict 2PL sin locks de lectura
READ COMMITTED	Strict 2PL, suelta locks de lectura al tiro
REPEATABLE READ	Strict 2PL
SERIALIZABLE	Strict 2PL + predicate locks

Paréntesis: PostgreSQL usa MVCC, no 2PL puro para lectores

PostgreSQL mantiene **versiones de cada fila** (MVCC). Los lectores ven un *snapshot* — **no toman S locks**, no bloquean a escritores ni se bloquean por ellos.

Nivel en Postgres	Implementación	Garantía real
READ UNCOMMITTED	tratado como READ COMMITTED	sin dirty reads
READ COMMITTED (default)	snapshot por sentencia	ve el último commit en cada query
REPEATABLE READ	snapshot por transacción	ve un estado consistente — pero NO es serializable
SERIALIZABLE	SSI (Serializable Snapshot Isolation)	detecta ciclos de dependencias y aborta

Volviendo a READ COMMITTED

Nivel	Versión de 2PL
READ UNCOMMITTED	Strict 2PL sin locks de lectura
READ COMMITTED	Strict 2PL, suelta locks de lectura al tiro
REPEATABLE READ	Strict 2PL
SERIALIZABLE	Strict 2PL + predicate locks

Podemos pedirle más a READ COMMITTED pidiendo locks de lectura por más tiempo.

Usando `SELECT ... FOR UPDATE` para garantizar lectura

En `READ COMMITTED` (el default), el patrón canónico para evitar lost updates es **bloquear las filas explícitamente al leerlas**:

```
BEGIN;  
  SELECT saldo FROM cuentas WHERE cid = 1 FOR UPDATE;  -- X lock  
  UPDATE cuentas SET saldo = saldo - 100 WHERE cid = 1;  
  UPDATE cuentas SET saldo = saldo + 100 WHERE cid = 2;  
COMMIT;
```

Variantes:

- `FOR SHARE` : S lock, permite múltiples lectores pero bloquea writers.
- `FOR UPDATE NOWAIT` : si no puede tomar el lock, **falla inmediatamente** en vez de esperar.
- `FOR UPDATE SKIP LOCKED` : ignora filas ya bloqueadas — patrón estándar para colas de trabajo.

Si usan SERIALIZABLE: hay que reintentar

SSI funciona detectando dependencias **al hacer commit**. Si rompió serializabilidad:

```
ERROR: could not serialize access due to read/write dependencies  
SQLSTATE: 40001
```

Esto **no es un bug**; es la garantía. Toda app que usa SERIALIZABLE debe envolver sus transacciones:

```
for attempt in range(MAX_RETRIES):  
    try:  
        with conn.transaction(isolation_level=SERIALIZABLE):  
            # ...lógica de la transacción...  
        break  
    except SerializationFailure:  
        time.sleep(backoff(attempt))  
else:  
    raise # excedió reintentos
```

Más isolation NO siempre es mejor

Un reporte read-only que no toca filas críticas, ejecutado en SERIALIZABLE, puede **abortar** porque SSI detectó una dependencia con otra transacción que sí escribió.

→ El usuario ve un error transitorio en una operación que **lógicamente** no necesitaba serializabilidad.

Subir el nivel **sin necesidad** → más aborts, más reintentos, peor experiencia. La pregunta correcta: *¿qué invariante de negocio estoy protegiendo?*

¿Cuándo usar cada nivel?

Caso	Nivel sugerido
Reportes con datos "casi al día"	READ COMMITTED
Lecturas con snapshot consistente (análisis multi-tabla)	REPEATABLE READ
Transferencias / reservas / inventario, con SELECT FOR UPDATE	READ COMMITTED
Invariantes complejos que cruzan filas (write skew)	SERIALIZABLE + retry loop

Regla práctica: el default (READ COMMITTED) + **FOR UPDATE** cuando corresponda **resuelve la mayoría de los casos**. SERIALIZABLE es una herramienta de último recurso, con costo real.

Comentarios sobre trabajo en el mundo real...

Granularidad de locks

Hasta ahora hablamos de "lock sobre el objeto X " sin precisar qué tan grande es X .

Granularidad	Ejemplo	Trade-off
Tabla	<code>LOCK TABLE ...</code>	bajo overhead, alta contención
Página	implícito en algunos engines	balance
Fila	<code>SELECT ... FOR UPDATE</code> , <code>UPDATE</code> en Postgres	alto overhead, baja contención
Predicado / rango	next-key locks (InnoDB), SSI (Postgres)	previene phantoms

Long-running transactions: el problema operacional #1

Una transacción que se queda abierta horas (porque la app abrió `BEGIN`, hizo una query, y olvidó cerrar) causa estragos:

- **Postgres MVCC:** mientras esa transacción exista, el `VACUUM` **no puede** liberar las versiones viejas → la tabla crece (**bloat**).
- **Replicación física:** el WAL se acumula esperando — el slot de replicación se llena y el primario se queda sin disco.
- **Locks retenidos:** `SELECT FOR UPDATE` mantenido bloquea a otras transacciones por todo ese tiempo.

Defensas:

- `idle_in_transaction_session_timeout` (cerrar la sesión).
- Monitoreo: `pg_stat_activity.state = 'idle in transaction'` con `xact_start` viejo.

Herramientas de diagnóstico en PostgreSQL

Vista / función	Para qué sirve
<code>pg_stat_activity</code>	Quién está conectado, qué query corre, en qué estado
<code>pg_locks</code>	Todos los locks otorgados y pendientes
<code>pg_blocking_pids(pid)</code>	Qué procesos están bloqueando a este pid
<code>EXPLAIN (ANALYZE, BUFFERS)</code>	Plan + I/O real
<code>SET statement_timeout = '5s'</code>	Límite por sentencia (defensivo)

```
-- ¿Quién está bloqueando a quién, ahora?  
SELECT pid, state, wait_event_type, wait_event, query  
FROM pg_stat_activity  
WHERE state <> 'idle';
```

Idempotencia: lo que la BD no garantiza

ACID protege la transacción **una vez recibida** — no protege contra **clientes que reintentan** sobre redes inestables:

```
Cliente → POST /transferir → red lenta → timeout  
Cliente reintenta → POST /transferir → ¡ahora se transfiere D0S veces!
```

La BD ejecutó dos transacciones, ambas perfectamente atómicas. El bug está afuera.

Patrón de defensa: idempotency key — el cliente envía un UUID único; el servidor verifica si ya procesó ese ID y, si sí, devuelve el resultado guardado en lugar de re-ejecutar.

Cierre del bloque transacciones

Concepto	Qué nos dio
2PL (clase anterior)	Garantiza serializabilidad en runtime
Strict 2PL	+ libera al commit → sin cascading aborts
Detección de deadlocks	Manejar el costo del 2PL
Niveles de aislamiento	Trade-off explícito; cuidado con write skew
SELECT FOR UPDATE	Patrón canónico para lost updates en READ COMMITTED
Diagnóstico	<code>pg_stat_activity</code> , <code>pg_locks</code> , timeouts

Falta una propiedad de ACID por discutir: **Atomicity** y **Durability** ante fallas. Antes vamos a ver **Triggers**.

Triggers

Más allá de las restricciones declarativas

Recordemos lo que el DBMS chequea **automáticamente**:

- PRIMARY KEY , UNIQUE , NOT NULL .
- FOREIGN KEY con CASCADE / SET NULL .
- CHECK (condición) simples.

Pero hay reglas que **no** son expresables como restricciones declarativas:

- "Si el stock baja de 10, registrar una alerta en otra tabla."
- "Cada vez que se inserta una venta, actualizar la tabla de totales."
- "Una transferencia siempre debe dejar un registro en Auditoría ."

→ Triggers.

¿Qué es un trigger?

Un **trigger** es un procedimiento almacenado que el DBMS ejecuta **automáticamente** en respuesta a un evento sobre una tabla (`INSERT` , `UPDATE` , `DELETE`).

Componentes:

1. **Evento**: qué dispara el trigger.
2. **Condición** (opcional): cuándo realmente actuar.
3. **Acción**: qué se ejecuta (un bloque PL/pgSQL, típicamente).

Modelo **ECA** (Event–Condition–Action).

Sintaxis general en PostgreSQL

```
-- 1. Definir la función que se ejecutará
CREATE OR REPLACE FUNCTION nombre_funcion()
RETURNS TRIGGER AS $$
BEGIN
    -- lógica usando NEW (fila nueva) y OLD (fila previa)
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- 2. Asociarla a una tabla y un evento
CREATE TRIGGER nombre_trigger
{ BEFORE | AFTER | INSTEAD OF } { INSERT | UPDATE | DELETE }
ON nombre_tabla
FOR EACH { ROW | STATEMENT }
EXECUTE FUNCTION nombre_funcion();
```

NEW y **OLD** son las filas implicadas (existen según el evento: **OLD** no existe en INSERT, **NEW** no existe en DELETE).

Ejemplo completo: validar stock

```
CREATE OR REPLACE FUNCTION check_stock()  
RETURNS TRIGGER AS $$  
BEGIN  
    IF NEW.stock < 0 THEN  
        RAISE EXCEPTION 'Stock no puede ser negativo (producto %)', NEW.id;  
    END IF;  
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;  
  
CREATE TRIGGER trg_stock  
BEFORE INSERT OR UPDATE ON Productos  
FOR EACH ROW  
EXECUTE FUNCTION check_stock();
```

Cualquier `UPDATE Productos SET stock = stock - 5` que dejaría stock negativo **falla** y la transacción se aborta.

Auditoria(id, tabla, accion, datos_viejos, datos_nuevos, ts) .

```
CREATE OR REPLACE FUNCTION log_cambios() RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO Auditoria(tabla, accion, datos_viejos, datos_nuevos)
    VALUES (TG_TABLE_NAME, TG_OP,
             CASE WHEN TG_OP <> 'INSERT' THEN OLD::TEXT END,
             CASE WHEN TG_OP <> 'DELETE' THEN NEW::TEXT END);
    RETURN COALESCE(NEW, OLD);
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_audit_cuentas
AFTER INSERT OR UPDATE OR DELETE ON Cuentas
FOR EACH ROW EXECUTE FUNCTION log_cambios();
```

TG_TABLE_NAME y TG_OP son variables especiales del trigger.

BEFORE vs. AFTER vs. INSTEAD OF

Modificador	Cuándo se ejecuta	Uso típico
BEFORE	Antes de la operación. Puede modificar NEW o cancelar la operación.	Validación, normalización (uppercase, defaults).
AFTER	Después de la operación, dentro de la misma transacción.	Auditoría, propagación a otras tablas.
INSTEAD OF	En lugar de la operación; sólo en vistas.	Hacer que una vista sea actualizable.

FOR EACH ROW vs. FOR EACH STATEMENT

```
UPDATE Productos SET precio = precio * 1.1 WHERE categoria = 'libros';
```

Si afecta **100 filas**:

- **FOR EACH ROW** : la función se llama **100 veces** (una por fila), con **OLD / NEW** distintas.
- **FOR EACH STATEMENT** : la función se llama **1 vez** para todo el statement.

Use **FOR EACH ROW** cuando necesite **decidir/actuar por fila**.

Use **FOR EACH STATEMENT** cuando una **única acción agregada** (e.g., "recompute caché") basta.

Triggers y transacciones

Un trigger se ejecuta **dentro de la misma transacción** que lo disparó.

- Si la sentencia que dispara el trigger está dentro de un `BEGIN ... COMMIT`, el trigger forma parte de esa transacción.
- Si el trigger falla (`RAISE EXCEPTION`), **toda la transacción aborta**.
- Los efectos del trigger son **atómicos** con la operación original: o ambos persisten o ninguno.

→ Esto los hace ideales para mantener invariantes que cruzan tablas.

Cuándo usar (y cuándo NO usar) triggers

Buenos usos:

- Validaciones complejas que un `CHECK` no expresa.
- Auditoría / logging automático.
- Mantener tablas derivadas o caches.

Malos usos:

- Lógica de negocio importante "escondida" en triggers (los hace difíciles de razonar y debuggear).
- Cadenas largas (`INSERT` dispara trigger que dispara otro trigger...).
- Performance crítica con `FOR EACH ROW` sobre updates masivos.

Regla: triggers para **invariantes**, no para **funcionalidad**.

Trampas operacionales de triggers

- **Orden de ejecución:** si hay varios triggers para el mismo evento sobre la misma tabla, Postgres los dispara **en orden alfabético por nombre**. Sorpresa garantizada.
Convención: prefijar con `01_`, `02_`.
- **Triggers recursivos:** un `UPDATE` dentro de un trigger de `UPDATE` puede disparar el mismo trigger → cuidado con loops infinitos.
- **TRUNCATE no dispara triggers FOR EACH ROW** (sólo los `FOR EACH STATEMENT` con evento `TRUNCATE`).
- **Replicación lógica:** los triggers no se disparan en el subscriber por default — su efecto vive sólo en el primario. Configurar con `session_replication_role` si hace falta.
- **Debugging:** no hay breakpoints. Se puede hacer algo usando `RAISE NOTICE` `'mensaje %', valor;` y revisar el server log.

Lecturas recomendadas

- Ramakrishnan & Gehrke, *Database Management Systems* (3ed):
 - **Cap. 17 — Concurrency Control** (locking, 2PL).
- Garcia-Molina, Ullman, Widom, *Database Systems: The Complete Book* (2ed), **Cap. 18.**
- Documentación PostgreSQL:
 - [Concurrency Control](#)
 - [CREATE TRIGGER](#)
- Berenson, Bernstein, Gray, Melton, O'Neil, O'Neil (1995). *A Critique of ANSI SQL Isolation Levels* — la fuente del concepto de **write skew** y la crítica al estándar.
- Fekete, Liarokapis, O'Neil, O'Neil, Shasha (2005). *Making Snapshot Isolation Serializable* — base teórica de SSI en PostgreSQL.